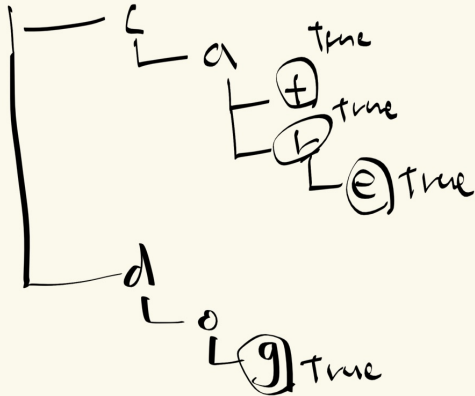


Trie (Prefix Tree)

: store string and enable to search it quickly

example: cat, car, cave, dog

(Root)



Code:

```
class TrieNode:
```

```
    def __init__(self):
        self.children = {}
        self.is_end = False
```

```
class Trie:
```

```
    def __init__(self):
        self.root = TrieNode()
```

```
    def insert(self, word): # TC : O(L)
```

```
        curr = self.root
```

```
        for ch in word:
```

```
            if ch not in curr.children:
                curr.children[ch] = TrieNode()
```

```
            curr = curr.children[ch]
```

```
        curr.is_end = True
```

```
    def search(self, word): # TC : O(L)
```

```
        curr = self.root
```

```
        for ch in word:
```

```
            if ch not in curr.children:
                return False
```

```
            curr = curr.children[ch]
```

```
        return curr.is_end
```

```

class TrieNode:
    def __init__(self):
        self.children = {}
        self.word = None

class Solution:
    def findWords(self, board: List[List[str]], words: List[str]) -> List[str]:
        # Trie 생성
        root = TrieNode()
        for word in words:
            node = root
            for ch in word:
                if ch not in node.children:
                    node.children[ch] = TrieNode()
                node = node.children[ch]
            node.word = word

        m, n = len(board), len(board[0])
        result = []

        def dfs(r, c, node):
            ch = board[r][c]

            if ch not in node.children:
                return

            nxt = node.children[ch]

            # 단어를 찾은 경우
            if nxt.word:
                result.append(nxt.word)
                nxt.word = None # 중복 방지

            board[r][c] = '#'

            for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
                nr, nc = r + dr, c + dc
                if (
                    0 <= nr < m
                    and 0 <= nc < n
                    and board[nr][nc] != '#'
                ):
                    dfs(nr, nc, nxt)

            board[r][c] = ch

            # 가지치기
            if not nxt.children and nxt.word is None:
                del node.children[ch]

        for i in range(m):
            for j in range(n):
                dfs(i, j, root)

        return result

```